Indicazioni Progetto Architettura e Sistemi Software Distribuiti

Obiettivo: programmare uno strumento per monitorare GPU e tutto ciò che interessa l'esecuzione per i modelli AI.

- Realizzare un sistema di monitoraggio per il consumo energetico e altri parametri di riferimento;
- Eventualmente realizzare dei benchmark. In primis realizzare l'architettura e poi una dashboard interattiva;
- Individuare le probe (software che sfruttano sensori già integrati che prelevano i dati di funzionamento della GPU e altri componenti);
- Raccogliere i dati delle probe in database per *time series* (come Prometeus);
- Utilizzare API o dashboard interattive per la visualizzazione (come Grafana) e poi esporre la nostra architettura tramite API;
- Testing su VM con AI di prova per testare aumenti e query alla variazione dei parametri.

Il tutto va documentato sul COME e soprattutto il PERCHÉ della scelta architetturale, per ogni componente, metrica, DB e infine della dashboard. Alla fine, produrre un report e un PowerPoint da discutere all'esame.

Testare su VM:

- Accesso VPN: http://192.168.240.225:1880/metrics (fare un filtro su DL380A);
- Accesso da terminale: ssh ubuntu@172.31.2.150 (pwd: assd20252026);
- Metriche: Power, voltage, ILO_Power_Supply;
- PDU=power delivery unity, ps=power supply;
- Se ci sono due (o più) power supply per macchina, è possibile sommarle.

Usare i seguenti **paper** per le fonti:

- Awesome Green Al.pdf
- Comparing energy consumption and accuracy in text classification inference.pdf
- Green Al techniques for reducing energy consumption in Al systems.pdf
- SCI Specifications.pdf

Le fasi del progetto

Fase 0 — Design architetturale (tutti insieme)
Prima di scrivere codice, decidete e motivate su carta:

- Stack: Prometheus (o alternativa) + Grafana (o dashboard custom) + eventuale DB time-series secondario (InfluxDB) se serve retention diversa.
- Quali metriche raccogliere oltre a quelle richieste (GPU utilization, temperatura, memory, se disponibili da nvidia-smi/DCGM sulla VM di test).
- Come sarà strutturata l'API che esporrete alla fine.
  Questo è il momento per leggere i 4 paper e fissare i criteri "green" che userete per giustificare le scelte (es. SCI - Software Carbon Intensity - come framework di riferimento per le metriche di sostenibilità).

Fase 1 — Probe discovery
Esplorare l'endpoint reale, capire che formato hanno le metriche, testare lo scraping via SSH sulla VM (ssh ubuntu@172.31.2.150), verificare cosa espone l'iLO e cosa invece va preso lato VM (es. GPU stats se la VM ha GPU passthrough).

Fase 2 — Pipeline dati / storage
Configurare Prometheus (o soluzione equivalente) per fare scraping periodico dell'endpoint, definire retention, eventuali regole di aggregazione (es. somma dei due power supply), esporre i dati in modo interrogabile.

Fase 3 — Dashboard + API
Costruire la dashboard interattiva (Grafana o dashboard custom) e un layer API REST che espone i dati aggregati della vostra architettura (non il grezzo di Prometheus, ma la vostra "vista" sul sistema).

Fase 4 — Testing con carico AI
Sulla VM, lanciare un modello di test (inference o training leggero), osservare come variano potenza/voltaggio, interrogare l'API/dashboard durante il carico e documentare i pattern osservati.

Fase 5 — Report + PPT
Documentare il "come" (implementazione) e soprattutto il "perché" di ogni scelta (componente, metrica, DB, dashboard) appoggiandovi ai paper. Preparare le slide per l'esame.

Come dividervi il lavoro (3 persone)

Un'unica architettura, tre responsabilità verticali che si incastrano:

Persona A — Probe & Data Collection

- Accesso VPN/SSH, esplorazione dell'endpoint /metrics, mapping delle metriche disponibili (Power, Voltage, ILO_Power_Supply, PDU/PS).
- Eventuale probe aggiuntiva lato VM per metriche GPU/AI (nvidia-smi, DCGM se disponibile).
- Documenta: quali probe esistono, perché sono state scelte quelle, limiti dei sensori.

Persona B — Storage & Pipeline (time-series DB)

- Setup di Prometheus (o alternativa), configurazione scraping, retention policy, regole di aggregazione (somma PS multipli).
- Gestione dello storico dati per i benchmark.
- Documenta: perché quel DB time-series, trade-off rispetto ad alternative.

Persona C — Dashboard & API

- Grafana (o dashboard custom) + layer API che espone l'architettura verso l'esterno.
- Design delle viste (per esame: cosa mostrare, come raccontare i dati).
- Documenta: scelte di visualizzazione, perché quella struttura di API.

Contenuti paper

SCI Specifications (Green Software Foundation) — è il framework di riferimento più operativo per voi. Vi dà una formula pronta da citare e applicare:

SCI = (O + M) / R dove O = energia × intensità carbonica della rete elettrica, M = emissioni incorporate dell'hardware (quota proporzionale a tempo e risorse usate), R = unità funzionale (es. per API call, per training run, per ora di monitoraggio).

Usatelo così: definite il vostro **software boundary** (probe + DB + dashboard + API), scegliete un'**unità funzionale R** sensata per il vostro caso (es. "per query alla dashboard" o "per ora di monitoraggio GPU"), e giustificate che misurate O (energia misurata via iLO/DCGM × carbon intensity della rete) ma probabilmente NON M (le VM di test difficilmente vi permettono di stimare le embodied emissions dell'hardware — potete dirlo esplicitamente come limite, seguendo l'approccio "measurement vs calculation" descritto nel documento).

Comparing energy consumption and accuracy (Zschache & Hartwig) — questo è il paper più simile metodologicamente al vostro progetto, perché fa esattamente quello che dovrete fare voi: misurare energia durante l'inferenza AI con uno strumento (loro usano CodeCarbon, che sfrutta NVML per la GPU e stime TDP per la CPU — lo stesso principio del vostro endpoint iLO/Prometheus).

Punti chiave da citare nel report:

- **Durata come proxy dell'energia**: trovano una relazione quasi lineare tra durata dell'inferenza e energia consumata (Energia ≈ (a + c·GPU)·Durata). Questo è oro per voi: se durante la Fase 4 (test con AI) non riuscite a misurare l'energia con precisione in tempo reale, potete usare la durata come proxy giustificato, citando questo risultato.
- **Il numero di GPU non riduce l'energia**, riduce solo la durata (a volte l'aumenta). Utile se testate scaling su più GPU.
- Modelli più efficienti energeticamente possono essere altrettanto o più accurati di modelli grandi — argomento utile se nella vostra fase di benchmark confrontate modelli AI di dimensioni diverse sulla VM.
- Limite metodologico onesto che potete riprendere: la stima CPU via TDP sovrastima il consumo reale (RAPL non sempre disponibile) — probabilmente avrete lo stesso problema con l'hardware HPE, quindi è un limite da dichiarare esplicitamente nel report.

Green AI techniques (Khan et al., 2026) — è una review sistematica, utile soprattutto per la Fase 0 (design) e per giustificare **quali metriche raccogliere oltre a Power/Voltage**:

- Dati concreti da citare: quantizzazione → fino al 50% di riduzione energetica; distillazione (DistilBERT) → 60% inferenza più veloce con 40% parametri in meno; TPU vs GPU → fino a 2.7× miglioramento performance/Watt.
- Framework a 3 livelli molto utile per strutturare il report: livello **algoritmico** (pruning, quantizzazione), livello **hardware** (GPU/TPU/accelerator), livello **infrastruttura/data center** (cooling, PUE, virtualizzazione). Potete mappare la vostra architettura su questi 3 livelli.
- PUE (Power Usage Effectiveness) è un concetto chiave che dovreste menzionare quando giustificate la scelta di misurare solo potenza IT e non l'overhead del data center (a meno che l'endpoint iLO non ve lo dia già).

Awesome Green AI — è una lista curata di tool, utile soprattutto per la sezione "stato dell'arte / strumenti alternativi" del report: potete citarla per giustificare perché avete scelto Prometheus+Grafana invece di, ad esempio, CodeCarbon o Scaphandre (che sono pensati per singola macchina/processo, non per infrastruttura fisica come i server HPE con iLO).

Come usarli operativamente nel report

Vi suggerisco questa mappatura paper → sezione del report:

Sezione del report	**Paper principale**	**Cosa citare**
Motivazione/contesto	Khan et al. + Awesome Green AI	perché il monitoraggio energetico AI è rilevante oggi
Scelta metriche (Power, Voltage, ILO_Power_Supply)	SCI Spec	formula O=E×I, giustificazione di cosa misurare
Scelta del DB time-series	— (motivate voi stessi: retention, query pattern)	eventualmente Khan et al. per il tema "reproducible measurement protocols"
Metodologia di misura durante i test AI	Zschache & Hartwig	uso della durata come proxy, limiti della stima TDP
Discussione limiti	Zschache & Hartwig + SCI Spec	esclusione embodied emissions, overstima CPU, mancanza PUE regionale
Raccomandazioni finali/policy	Khan et al.	checklist di reporting energetico, model cards con sezione energia

Un consiglio pratico per dividervi anche questa parte:

- la **Persona A (probe)** può occuparsi di leggere SCI Spec + Zschache&Hartwig per definire cosa e come misurare;
- la **Persona B (storage)** può usare Khan et al. per giustificare granularità/retention dei dati;
- la **Persona C (dashboard/API)** può ispirarsi a come Zschache&Hartwig e Khan et al. presentano i loro risultati (Fig. 2 del primo paper, con trade-off accuracy-energia, è un ottimo modello per una vostra visualizzazione).